UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Ingeniería en Sistemas de Información y Ciencias de la Computación

PROYECTO FINAL DE ALGORITMOS

PROYECTO 18 · VARIANTE B
SISTEMA DE CONTROL DE PEAJE

Curso: Algoritmos

Catedrático: Ing. Mag. Jhensser Aguirre

Estudiante: JAQUELINE JIMENA HERNANDEZ RAXON

Carné: 2294-26-13355

Sección: A · Segundo semestre 2026

Índice

1. Introducción
2. Descripción y planteamiento del problema
3. Objetivos
4. Alcance y supuestos de diseño
5. Usuarios, entradas, procesos y salidas
6. Requerimientos funcionales
7. Reglas de negocio y validaciones
8. Casos límite y situaciones de error
9. Descomposición modular
10. Diccionario básico de datos
11. Diseño del menú
12. Diagrama general del sistema
13. Pseudocódigo de los algoritmos principales
14. Estructuras y algoritmos utilizados
15. Simulación por ciclos y métricas
16. Reportes del sistema
17. Archivos y persistencia
18. Matriz de pruebas
19. Manual breve de usuario
20. Conclusiones
21. Recomendaciones y mejoras futuras
22. Organización de entregas
23. Preparación para la defensa

1. Introducción

El presente proyecto desarrolla un sistema de consola en C++17 para simular la operación básica de una estación de peaje. El sistema administra carriles, registra el ingreso de vehículos, los clasifica por categoría, los coloca en colas, procesa la atención por ciclos, calcula cobros y genera estadísticas. La solución se plantea de forma modular y utiliza arreglos, estructuras, ciclos, condicionales, búsquedas, ordenamiento manual y archivos de texto, de acuerdo con los requisitos técnicos indicados en la guía del Proyecto Final de Algoritmos 2026.

2. Descripción y planteamiento del problema

Una estación de peaje recibe vehículos de diferentes categorías y debe distribuirlos entre carriles disponibles. Si no existe un control adecuado, pueden presentarse filas desbalanceadas, cobros incorrectos, duplicidad de registros o pérdida de información. Se propone un sistema que asigne cada vehículo al carril habilitado con menor cantidad de vehículos en espera y atienda como máximo un vehículo por carril en cada ciclo de simulación.

3. Objetivos

Objetivo general: diseñar e implementar un sistema de consola que controle el ingreso, clasificación, espera, atención y cobro de vehículos en una estación de peaje.

Objetivos específicos:

• Registrar vehículos mediante una placa y una categoría.

• Distribuir los vehículos entre los carriles habilitados.

• Simular la atención por ciclos, respetando el orden de llegada en cada cola.

• Calcular tarifas, recaudación y tiempos de espera.

• Permitir búsquedas por placa e identificador.

• Generar reportes generales, por categoría y por carril.

• Guardar y recuperar el historial mediante archivos de texto.

4. Alcance y supuestos de diseño

El sistema funciona en consola, utiliza C++17 y está pensado para abrirse en CLion mediante CMake. Incluye tres carriles iniciales y tres categorías: motocicleta, automóvil y transporte pesado. Las tarifas de ejemplo son Q10.00, Q20.00 y Q35.00, respectivamente; pueden modificarse desde el menú. Estos importes, la cantidad inicial de carriles y la regla de un vehículo atendido por carril en cada ciclo son decisiones de diseño para hacer ejecutable la simulación; la guía del docente no fija tarifas ni cantidades concretas.

El sistema no utiliza interfaz gráfica, base de datos ni bibliotecas externas. La persistencia implementada guarda el historial de vehículos atendidos. Las colas pendientes permanecen durante la ejecución, pero no se restauran después de cerrar el programa; esta limitación se deja identificada como mejora para una siguiente versión.

5. Usuarios, entradas, procesos y salidas

Usuario: operador de la estación de peaje.

Entradas: placa del vehículo, categoría, opción del menú y nuevas tarifas cuando se configure el sistema.

Procesos: validación de placa; búsqueda del carril con menor cola; inserción en cola; avance del ciclo; retiro del primer vehículo de cada carril; cálculo de tarifa y espera; actualización de contadores; búsqueda; ordenamiento y generación de reportes.

Salidas: confirmación de ingreso, carril asignado, estado de colas, resultado de búsquedas, tarifa cobrada, recaudación, cantidad atendida y espera promedio.

6. Requerimientos funcionales

RF-01. Mostrar un menú principal con opciones numeradas.

RF-02. Registrar un vehículo solicitando placa y categoría.

RF-03. Rechazar placas vacías, mayores de 12 caracteres o duplicadas en espera.

RF-04. Asignar el vehículo al carril habilitado con menor cola.

RF-05. Mantener el orden FIFO (primero en entrar, primero en salir) en cada carril.

RF-06. Procesar ciclos de atención.

RF-07. Cobrar la tarifa correspondiente a la categoría al momento de la atención.

RF-08. Consultar el contenido y estado de los carriles.

RF-09. Buscar vehículos por placa y por ID.

RF-10. Modificar las tarifas para atenciones futuras.

RF-11. Ordenar manualmente el historial por ciclos de espera.

RF-12. Generar reportes generales, por categoría y por carril.

RF-13. Guardar el historial en un archivo de texto.

RF-14. Cargar el historial guardado al iniciar el programa.

7. Reglas de negocio y validaciones

RV-01. Solo se aceptan las categorías 1, 2 y 3.

RV-02. La placa debe tener entre 1 y 12 caracteres y no puede estar repetida entre vehículos que esperan.

RV-03. Un vehículo solo puede ingresar a un carril habilitado con espacio disponible.

RV-04. La asignación se realiza al carril con menor cantidad de vehículos en espera; en empate se elige el primero.

RV-05. Cada carril atiende como máximo un vehículo por ciclo.

RV-06. La atención respeta FIFO dentro de cada carril.

RV-07. La tarifa se toma de la categoría del vehículo cuando se atiende.

RV-08. Las tarifas deben ser mayores que cero y no superar Q10,000.

RV-09. No se procesa un vehículo si el historial alcanzó su capacidad máxima.

RV-10. Las opciones del menú deben estar dentro del rango permitido.

8. Casos límite y situaciones de error

• Ingresar una opción de menú con letras o fuera del rango: se limpia la entrada y se solicita nuevamente.

• Registrar una categoría menor que 1 o mayor que 3: se rechaza y se vuelve a solicitar.

• Ingresar una placa ya presente en una cola: se rechaza.

• Intentar registrar cuando todos los carriles están llenos o deshabilitados: se informa que no hay espacio.

• Buscar una placa o ID inexistente: se muestra un mensaje sin alterar los datos.

• Intentar cargar un archivo inexistente: se inicia una sesión nueva.

• Cargar un archivo con formato incorrecto: no se reemplaza el historial actual.

• Alcanzar el máximo de registros históricos: se detiene la atención antes de retirar vehículos de las colas.

9. Descomposición modular

inicializarCarriles(): prepara los carriles y sus contadores.

leerEntero() y leerTarifa(): validan los datos numéricos.

buscarCarrilDisponible(): encuentra el carril con menor cola.

agregarVehiculo(): valida y registra un ingreso.

procesarCiclo(): avanza el tiempo simulado y atiende vehículos.

registrarAtencion(): calcula y acumula los resultados de una atención.

mostrarColas(): presenta el estado de cada carril.

buscarPorPlaca() y buscarPorId(): realizan búsquedas secuenciales.

cambiarTarifas(): actualiza los importes.

ordenarHistorialPorEspera(): aplica burbuja (bubble sort).

reporteGeneral(), reportePorCategoria() y reportePorCarril(): generan estadísticas.

guardarDatos() y cargarDatos(): administran el archivo de texto.

menuReportes() y main(): controlan la navegación del sistema.

10. Diccionario básico de datos

Vehiculo: estructura que contiene id (entero), placa (cadena), categoria (entero 1–3), cicloIngreso (entero), ciclosEspera (entero) y activo (booleano).

Carril: estructura con numero, habilitado, cola de vehículos, frente, final, cantidad, atendidos y recaudacion.

Registro: estructura histórica con ID, placa, categoría textual, carril, ciclo de ingreso, ciclo de atención, espera y tarifa.

tarifas[3]: arreglo de tarifas por categoría.

carriles[4]: arreglo de carriles; se habilitan tres al inicio.

cola[100]: arreglo circular de vehículos por carril.

pendientes[500]: arreglo de control de vehículos ingresados durante la sesión.

historial[500]: arreglo de atenciones registradas.

cicloActual: contador entero de ciclos simulados.

totalRecaudado: acumulador decimal de cobros.

11. Diseño del menú

1. Registrar ingreso de vehículo.

2. Procesar siguiente ciclo.

3. Consultar carriles y colas.

4. Buscar por placa.

5. Buscar por ID.

6. Configurar tarifas.

7. Ordenar historial por espera.

8. Consultar reportes.

9. Guardar datos.

0. Guardar y salir.

El submenú de reportes ofrece: reporte general, reporte por categoría y reporte por carril.

12. Diagrama general del sistema

INICIO

↓

Inicializar carriles y cargar historial

↓

Mostrar menú principal

↓

Leer y validar opción

■■ Registrar vehículo → validar placa/categoría → asignar carril → volver al menú

■■ Procesar ciclo → avanzar ciclo → atender primero de cada carril → actualizar cobros

■■ Consultar / buscar → mostrar resultados → volver al menú

■■ Configurar tarifas → validar importes → volver al menú

■■ Reportes / ordenar → calcular o clasificar → volver al menú

■■ Guardar y salir → FIN

13. Pseudocódigo de los algoritmos principales

ALGORITMO RegistrarVehiculo

Leer placa

SI placa inválida o repetida ENTONCES

Mostrar error y terminar proceso

FIN SI

Leer categoría

Validar que categoría esté entre 1 y 3

Buscar carril habilitado con menor cantidad en cola

SI no existe carril disponible ENTONCES

Mostrar falta de espacio

SINO

Crear vehículo con ID nuevo y ciclo actual

Insertar al final de la cola circular del carril

Mostrar ID y carril asignado

FIN SI

FIN ALGORITMO

ALGORITMO ProcesarCiclo

Incrementar ciclo actual

PARA cada carril habilitado

SI existe vehículo en la cola Y hay espacio en historial

Extraer el vehículo ubicado al frente

Calcular espera = ciclo actual - ciclo de ingreso

Obtener tarifa según categoría

Registrar atención y actualizar acumuladores

FIN SI

FIN PARA

FIN ALGORITMO

ALGORITMO AsignarCarril

menor ← capacidad máxima + 1

elegido ← -1

PARA cada carril

SI está habilitado Y cantidad < capacidad Y cantidad < menor

menor ← cantidad

elegido ← posición del carril

FIN SI

FIN PARA

RETORNAR elegido

FIN ALGORITMO

14. Estructuras y algoritmos utilizados

Se utilizan arreglos estáticos para almacenar carriles, colas, vehículos pendientes e historial. Cada cola es circular: frente identifica el próximo vehículo a atender y final la última posición ocupada. Esto permite insertar y retirar sin desplazar todos los elementos.

La búsqueda por placa y por ID es secuencial: revisa los registros uno por uno hasta encontrar coincidencia. El ordenamiento del historial utiliza el método burbuja, comparando registros adyacentes e intercambiándolos cuando el tiempo de espera está en orden incorrecto. No se utiliza std::sort.

15. Simulación por ciclos y métricas

Un ciclo representa una ronda de atención. En cada ciclo se revisan los carriles habilitados y se atiende como máximo un vehículo del frente de cada cola no vacía. Por ello, un ciclo puede atender cero, uno o varios vehículos, dependiendo de las colas.

La espera se calcula mediante: ciclos de espera = ciclo de atención – ciclo de ingreso. La recaudación total es la suma de las tarifas de todos los vehículos atendidos. La espera promedio es la suma de ciclos de espera dividida entre la cantidad de vehículos atendidos; si no hay atenciones, se presenta cero.

16. Reportes del sistema

Reporte general: ciclo actual, vehículos atendidos, recaudación total, vehículos en espera y espera promedio.

Reporte por categoría: cantidad atendida e ingresos generados por motocicletas, automóviles y transporte pesado.

Reporte por carril: cantidad atendida, recaudación y vehículos que permanecen en espera en cada carril.

Estos tres reportes permiten observar el volumen de atención, las categorías con mayor aporte y la carga operativa de cada carril.

17. Archivos y persistencia

El programa utiliza ofstream para guardar y ifstream para recuperar información en peaje_datos.txt. Se conserva el ciclo actual, el siguiente identificador y el historial de atenciones. El archivo utiliza campos separados por el carácter |. Al cargar, se verifica el encabezado, la cantidad de registros y la estructura de cada línea antes de sustituir el historial.

Nota de alcance: la versión incluida no persiste las colas pendientes ni las tarifas personalizadas. Al reiniciar, las colas comienzan vacías y se aplican las tarifas iniciales del código. Para una versión ampliada, se recomienda guardar también configuración y vehículos en espera.

18. Matriz de pruebas

Prueba | Entrada/acción | Resultado esperado

P01 | Registrar placa MOTO01, categoría 1 | Se asigna a carril disponible y se genera ID.

P02 | Registrar de nuevo MOTO01 mientras espera | Se rechaza por placa duplicada.

P03 | Seleccionar categoría 4 | Se solicita una categoría válida.

P04 | Ingresar letras en opción de menú | Se informa error y se vuelve a solicitar.

P05 | Registrar vehículos en varios carriles | Se asignan buscando la cola más corta.

P06 | Procesar un ciclo con colas ocupadas | Se atiende como máximo uno por carril.

P07 | Buscar placa existente | Se muestra estado o registro atendido.

P08 | Buscar placa inexistente | Se informa que no fue encontrada.

P09 | Cambiar tarifa a Q25 | Se acepta y se aplica a atenciones posteriores.

P10 | Consultar reporte por categoría | Se muestran cantidades e ingresos.

P11 | Ordenar historial | Se presenta espera de menor a mayor.

P12 | Guardar y reiniciar | Se recupera el historial guardado.

P13 | Cargar archivo inexistente | Se inicia una sesión nueva sin cerrar el programa.

P14 | Historial lleno | No se extraen vehículos adicionales de las colas.

19. Manual breve de usuario

1. Abrir en CLion la carpeta que contiene CMakeLists.txt.

2. Esperar la configuración de CMake y ejecutar Proyecto18_Peaje.

3. Registrar vehículos con placas distintas y seleccionar su categoría.

4. Consultar las colas para observar la asignación.

5. Seleccionar “Procesar siguiente ciclo” para atender los vehículos.

6. Consultar los reportes para revisar cobros y estadísticas.

7. Guardar los datos antes de cerrar; la opción 0 también guarda automáticamente.

8. El archivo peaje_datos.txt se genera en el directorio de ejecución.

20. Conclusiones

El proyecto permite representar mediante algoritmos el funcionamiento básico de una estación de peaje. La distribución por menor cola ayuda a balancear los ingresos entre carriles y las colas circulares conservan el orden de atención. La simulación por ciclos permite calcular espera, volumen atendido y recaudación. La modularidad facilita probar y explicar cada responsabilidad del programa.

La persistencia del historial permite conservar resultados entre ejecuciones, aunque la versión actual no restaura vehículos pendientes ni tarifas configuradas. Esta limitación debe considerarse al evaluar el alcance implementado.

21. Recomendaciones y mejoras futuras

• Persistir también las colas pendientes, el estado de los carriles y las tarifas.

• Permitir habilitar o deshabilitar carriles desde el menú con validación de que no queden vehículos sin asignación.

• Agregar fecha y hora real a cada registro.

• Incorporar reportes exportables y filtros por rango de ciclos.

• Probar el sistema con mayor volumen de vehículos y documentar los resultados.

• Ajustar las tarifas y las reglas operativas si el catedrático proporciona valores oficiales.

22. Organización de entregas

ENTREGA 1 – Análisis y núcleo algorítmico: problema, alcance, entradas/procesos/salidas, requerimientos, validaciones, casos límite, módulos, diagramas, pseudocódigo, diccionario de datos, menú, pruebas previstas y prototipo inicial.

ENTREGA 2 – Sistema funcional: correcciones de la primera entrega, modularización, arreglos, búsquedas por dos criterios, ordenamiento manual, contadores, acumuladores, reportes y pruebas normales, de borde y de error.

ENTREGA FINAL – Sistema completo: menú, funciones, estructuras, búsquedas, ordenamiento, archivos de texto, reportes, validaciones, documentación, manual y defensa individual. La guía exige que el estudiante pueda explicar y modificar su código.

23. Preparación para la defensa

El estudiante debe poder explicar: por qué se asigna al carril con menor cola; cómo funcionan frente y final en una cola circular; por qué se atiende un vehículo por carril en cada ciclo; cómo se calcula la espera; cómo se actualizan los acumuladores; cómo se realiza la búsqueda secuencial; cómo funciona el ordenamiento burbuja; y cómo se guardan y validan los registros del archivo.

Durante la defensa se recomienda demostrar un ingreso, una búsqueda, el procesamiento de un ciclo, un reporte, el ordenamiento y el guardado de datos. También debe poder modificar una tarifa o una validación pequeña y volver a compilar.

Anexo A. Código fuente

El archivo main.cpp se entrega por separado y dentro del paquete ZIP. Está configurado para C++17 y compila con CMake.